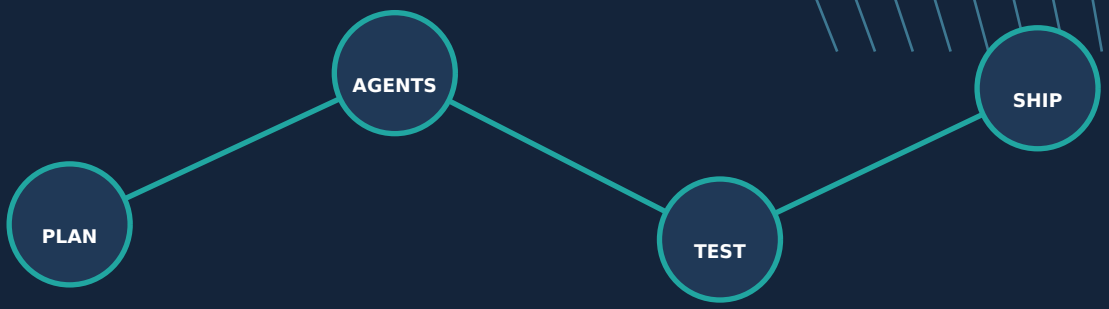
CAPSTONE PLAYBOOK

FinAlly

AI Trading Workstation

A step-by-step guide to building a multi-agent capstone with planning files, coding agents, testing and Docker.



Prepared from the three uploaded Vibe Engineering lecture recordings, the matching course notes, and the public project specification.

Version 1.0 • October 2026

How to use this guide

This document is organized as a build sequence rather than a lecture transcript. Start at the project contract, establish boundaries, then build the app in vertical slices. Use a coding agent to implement each slice, inspect the diff, and run the stated checks before moving on.

> Source-coverage note: The uploaded recordings were inspected through regular screen samples and cross-checked against the matching Vibe Engineering course pages and the published FinAlly project specification. A complete speech-to-text transcript was not available in this environment. Details that were not visible or documented are not presented as verified quotations from the lecturer.

What the capstone is

FinAlly (Finance Ally) is a Bloomberg-inspired workstation for simulated trading. It shows a watchlist, continuously updating prices, a virtual portfolio, P&L visualizations, and an AI assistant that can interpret portfolio questions and propose or apply portfolio actions within the simulation. The purpose of the project is to demonstrate coordinated AI-assisted software development—not to connect to a real brokerage.

Primary outcomes

- A working full-stack web application with a dark, data-dense terminal UI.
- A Python API that owns all business rules, price updates, portfolio calculations, and persistence.
- A TypeScript frontend that consumes agreed REST endpoints and a one-way Server-Sent Events (SSE) price stream.
- An LLM chat workflow that returns structured data and sends every requested action through server-side validation.
- A repeatable, single-container Docker build and an automated verification checklist.

> Safety boundary: Keep this project virtual. Use the built-in market simulator by default. Do not connect a brokerage account, add real order execution, or treat AI-generated commentary as financial advice. The project demonstrates agentic programming and software architecture.

Contents

01 Project scope and acceptance criteria
02 System architecture
03 Technology stack
04 Prerequisites and workspace setup
05 Project structure
06 Write the plan before coding
07 Organize AI coding agents
08 Build the backend and data model
09 Price simulation and SSE streaming
10 Add the LLM assistant safely
11 Build the frontend terminal
12 Integrate and run the application
13 Testing, debugging and definition of done
14 Useful course techniques: commands, sub-agents, hooks and plugins
15 Sandboxing, remote work and advanced agent workflows
16 Troubleshooting reference
17 Build checklist and milestones
18 Glossary and sources

> Suggested order: complete Sections 1-7 before parallel implementation. Then build and verify one backend/API slice, one UI slice, the LLM workflow, and finally Docker integration. Do not ask multiple agents to edit the whole repository at once.

01 Project scope and acceptance criteria

Core features

- Watchlist: start with ten seeded ticker symbols: AAPL, GOOGL, MSFT, AMZN, TSLA, NVDA, META, JPM, V and NFLX; show latest simulated price and change.
- Virtual portfolio: begin with $10,000 virtual cash; allow market-style buy/sell operations against simulated prices.
- Live-looking price updates: an in-process simulator changes prices; the backend broadcasts updates through SSE.
- Portfolio visualizations: positions table, portfolio heatmap/treemap and value-over-time chart.
- AI assistant: explain portfolio state, summarize concentration/P&L, and propose watchlist changes or simulated trades through a controlled JSON response.
- Persistence: SQLite keeps positions, watchlist, trades, chat history and snapshots between restarts.
- One-command startup: a Dockerized app serves the frontend and API on port 8000.

Non-goals for the first version

- No login or multi-user account management; treat the first version as a local single-user app.
- No options, limit orders, market depth, short selling, exchange connectivity, or real-money execution.
- No microservice fleet. Keep the design lean: one backend app, one SQLite file, one container.
- No assumption that the simulated prices represent current market prices or are suitable for investment decisions.

Definition of done

Area	Acceptance check
Startup	Docker starts the application and browser can open http://localhost:8000.
Market data	Price stream updates; frontend reconnects after an interrupted SSE connection.
Watchlist	Add/remove a ticker; duplicates and invalid tickers are handled.
Virtual trading	A valid buy updates cash and holdings; invalid buys/sells are rejected clearly.
Persistence	A restart preserves stored portfolio/watchlist/trade history.
AI chat	A message returns text plus validated structured actions; malformed output cannot mutate the portfolio.
Tests	Unit and end-to-end tests cover core paths and common failure conditions.

02 System architecture

The architecture in the course specification deliberately favors a single container and one browser origin. The frontend is exported as static files; FastAPI serves those files and the API. This reduces local setup and avoids having to configure cross-origin requests between separate frontend/backend servers.

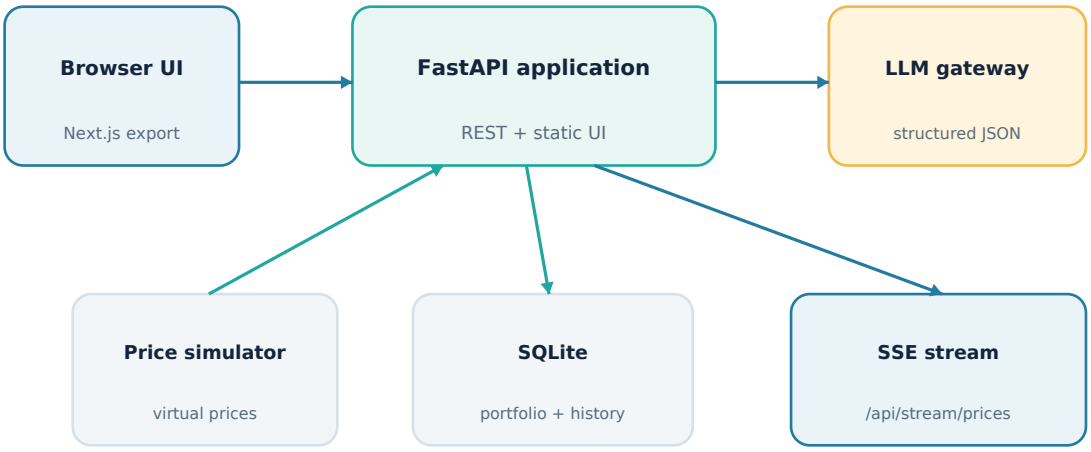


Figure 1. Application architecture (conceptual, based on the published project plan).

Main responsibilities

Component	Owns	Must not do
Frontend (Next.js)	UI state, charts, forms, SSE subscription, chat display.	It must not authoritatively calculate or persist trades.
FastAPI backend	REST endpoints, validation, portfolio calculations, LLM coordination, static file serving.	It must not trust AI output without validation.
Price simulator + cache	Generate virtual prices and hold latest/previous/timestamp values.	It must not access brokerage APIs.
SQLite	Persistent ledger, watchlist, virtual cash, history and chat messages.	It must not be directly edited by the frontend.
LLM adapter	Turn user requests plus portfolio context into schema-constrained proposals.	It must not bypass server-side business rules.
SSE stream	Push price events from server to browser.	It is one-way data delivery, not the trade-command channel.

03 Technology stack

Layer	Course-specification choice	Why it is used
Frontend	Next.js + TypeScript + Tailwind CSS, static export	Component UI and type-checked frontend; export makes it easy for FastAPI to serve the app.
Backend	Python + FastAPI + uv	Typed REST endpoints, asynchronous tasks and simple Python tooling.
Database	SQLite	Single-file persistence; no separate database server.
Streaming	Server-Sent Events (SSE) + browser EventSource	One-way server-to-browser price updates; simpler than WebSockets for this use.
LLM access	LiteLLM → OpenRouter; the course plan shows a Cerebras-backed route and structured outputs	Keep model access behind one adapter so provider changes do not rewrite business logic.
Market data	Built-in geometric-Brownian-motion-style simulator by default; optional data-provider adapter in the spec	Lets the app demonstrate streaming without needing an external market-data key.
Container	Multi-stage Docker build; Node build stage + Python runtime stage	One image contains the static UI and API; port 8000 is the single entry point.
Testing	pytest for backend; frontend checks; Playwright-style end-to-end tests	Verify rules, UI behavior and the complete journey.

> Version note: The model/provider pairing and developer-tool UX shown in course material can change over time. Keep the LLM provider and coding-agent commands configurable; check current vendor documentation before installing or copying provider-specific commands. The core design does not depend on a particular model brand.

Why SSE, not WebSockets?

Price updates primarily travel in one direction: server to browser. The browser opens an EventSource connection to `/api/stream/prices`; the backend emits event records; the UI updates cells and charts. User actions such as buying virtual shares or sending a chat message use ordinary REST requests.

04 Prerequisites and workspace setup

Install or verify the following tools before asking the agent to scaffold the project. Use the versions specified by the starter repository where available.

Tool	Purpose	Verify
Git	Version history and safe checkpoints.	git --version
Node.js + npm	Build the Next.js static frontend.	node --version; npm --version
Python 3.12	Run FastAPI backend.	python --version
uv	Manage Python packages and lockfile.	uv --version
Docker Desktop/Engine	Build and run the single container.	docker --version; docker info
VS Code or similar IDE	Inspect diffs and run coding-agent workflows.	Open the project folder.
Coding agent	Agentic implementation and review.	Confirm it can read/write this repo and execute approved commands.
LLM API key (optional initially)	Power the live chat workflow.	Use mock mode until the backend path works.

Start from the course skeleton if you have it

The lecture demonstrates a deliberately sparse starter repository with `CLAUDE.md`, `planning/PLAN.md`, empty frontend/backend/test areas and the course-specific skill. If your instructor shared that repo, use it so your file names match the lesson. If you do not have the skeleton, create the directories below and let your coding agent scaffold each project.

Create a fresh working directory (manual fallback)

```
mkdir finally
cd finally
git init
mkdir planning frontend backend db test scripts
mkdir -p .claude/commands .claude/skills
```

> Before running any autonomous coding agent, create a Git checkpoint and keep secrets out of version control. Start with narrow tool permissions, inspect file changes, and approve package installs or destructive commands deliberately.

05 Project structure

Treat this as the target shape, not as a claim that every file exists in the starter repo. Allow the implementation agent to select internal component files, but keep the top-level ownership boundaries stable.

```
finally/
├── CLAUDE.md                # Claude Code instructions; points to PLAN.md
├── AGENTS.md                # optional shared rules for other coding agents
├── .env.example             # safe template; never commit actual secrets
├── .gitignore
├── planning/
│   ├── PLAN.md              # single source of truth for scope and architecture
│   ├── API_CONTRACT.md      # request/response schema agreed by all agents
│   ├── TASKS.md             # milestones, owners, acceptance checks
│   └── reviews/              # review reports and decisions
├── frontend/                 # Next.js + TypeScript + Tailwind static export
│   ├── src/components/       # Watchlist, Charts, Portfolio, Chat, TradeBar
│   └── src/lib/              # API client, types, SSE hook
├── backend/                  # Python uv project + FastAPI app
│   ├── app/                  # routes, services, models, database layer
│   ├── db/                   # schema / seed / initialization
│   └── tests/                # backend unit tests
├── db/                       # runtime volume mount; database file is ignored
├── test/                     # end-to-end tests and test orchestration
├── scripts/                  # start/stop helpers (PowerShell + shell if needed)
├── Dockerfile                # multi-stage build serving app on :8000
└── docker-compose.yml        # optional convenience wrapper
```

File ownership contract

Path	Owner	Contract
planning/PLAN.md	Human + lead agent	Requirements, stack, boundaries and acceptance criteria.
planning/API_CONTRACT.md	Lead + backend + frontend agents	Endpoint paths, schemas, status codes and error shape. Freeze before parallel coding.
frontend/	Frontend agent	UI only; calls /api/* endpoints; no direct database access.
backend/	Backend agent	API, domain rules and persistence; defines truth for cash/positions.
backend/app/market/ (or equivalent)	Market-data agent	Price simulator, provider abstraction and SSE source.
Backend chat/LLM module	LLM agent	Prompt, schema-constrained output, safe action validation handoff.
test/ + read-only review	QA/reviewer agent	Tests and findings; no broad unrelated refactors.

06 Write the plan before coding

The most important practice in the lectures is to make `planning/PLAN.md` the single source of truth. A short root instruction file points agents at it on every turn. This avoids letting one agent invent an API, another invent a database shape, and a third build a UI against different assumptions.

Step 1 - write the project vision

Write one clear paragraph describing what the product does, who uses it and what is out of scope. Iterate with your coding agent in question-and-answer mode until the acceptance criteria are specific enough to test.

Step 2 - add core requirements to PLAN.md

```
# FinAlly - Finance Ally

## Vision
A local, single-user workstation for simulated market prices, a virtual portfolio,
portfolio visualizations, and an LLM assistant. No real brokerage integration.

## Required features
- Default watchlist with 10 tickers and virtual starting cash of 10000.
- Simulated prices and SSE updates.
- Buy/sell virtual shares with server-side validation.
- Portfolio positions, P&L, heatmap, and value history.
- AI chat that returns schema-constrained proposals.
- SQLite persistence and Docker startup on port 8000.

## Non-goals
- No real orders, brokerage credentials, or real-money transactions.
- No login or multi-user authorization in v1.
- No microservice split.

## Architecture boundaries
- frontend/: Next.js static export; talks only to same-origin /api/* routes.
- backend/: FastAPI, all domain rules, persistence, LLM and SSE.
- planning/: requirements and contracts shared by agents.

## Acceptance criteria
- Invalid trade requests are rejected without changing state.
- Prices stream and UI reconnects.
- Data persists across restarts.
- Mock mode makes tests deterministic.
- Docker starts on port 8000.
```

Step 3 - tell Claude Code to load the plan

```
# CLAUDE.md

# FinAlly - Finance Ally

Project-wide requirements, architecture, and rules live in planning/.
The plan is the single source of truth; read it before doing work.

@planning/PLAN.md

Before editing: restate your task, identify owned files, and check the API contract.
After editing: run relevant tests and summarize changed files and remaining risks.
```

Step 4 - support other coding tools

If you also use Codex CLI or another coding agent, add an `AGENTS.md` with equivalent repository rules and a direct instruction to read `planning/PLAN.md`. Keep one canonical specification; avoid maintaining conflicting copies of the requirements.

> Do not launch three implementation agents while the plan is still vague. The plan and the API contract are how the agents share memory; parallelism works only after ownership boundaries are clear.

07 Organize AI coding agents

The workflow is best understood as a manager plus specialized workers, not a crowd of agents all editing the same files. A sub-agent can be useful for an isolated task such as reviewing a plan or inspecting tests; independent coding agents are useful when their directories and interfaces do not overlap.

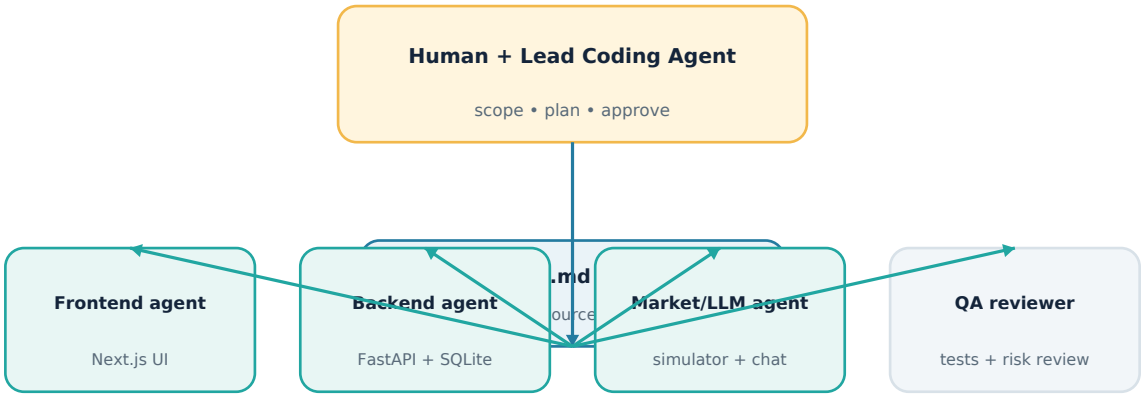


Figure 2. Agent roles and their shared contract.

Recommended ownership model

Role	Deliverable	Boundary
Lead / planner (you supervise)	PLAN.md, API contract, milestones, integration decisions.	Do not let agent output silently redefine requirements.
Backend engineer	FastAPI routes, services, SQLite setup and backend tests.	Own backend/; publish schemas before the UI depends on them.
Market-data engineer	Simulated price generator, cache, SSE route and reconnection assumptions.	Own market-data module; no frontend or LLM prompt edits.
Frontend engineer	Terminal dashboard, components, chart state, EventSource hook.	Own frontend/; consume published API types.
LLM integration engineer	Chat prompt, structured response models, schema validation and mock mode.	Propose actions only; domain service validates and applies.
QA / reviewer sub-agent	Read-only review, unit/E2E tests and severity-ranked findings.	Report findings; avoid unsolicited whole-project rewrites.

A safe sequence

1. Ask one planning agent to challenge the scope and identify ambiguity. Decide which suggestions to accept.
2. Agree the API contract and data types before frontend/backend agents work in parallel.
3. Implement the backend skeleton and health endpoint; test it before adding streaming or chat.
4. Have frontend and backend agents work in their own directories against the contract.
5. Add the LLM integration only after the deterministic virtual portfolio paths pass tests.
6. Run an independent reviewer, fix high-severity issues, then use the human to judge whether the app meets the goal.

08 Build the backend and data model

The backend is the source of truth. The browser asks for state and actions; FastAPI validates them, updates SQLite, and returns the canonical result. Keep business logic in services instead of scattering it across route handlers.

Suggested domain modules

```
backend/app/
├── main.py             # app creation, lifespan, static files
├── routes/             # health, market, portfolio, watchlist, chat endpoints
├── services/           # portfolio rules, market service, LLM coordinator
├── db/                 # connection, schema creation, seeding
├── models/             # request/response models (e.g. Pydantic)
└── tests/              # unit tests for services and routes
```

SQLite tables (v1)

Table	Purpose / key fields
users_profile	Single default profile and cash_balance (starts at 10000 virtual dollars).
watchlist	Ticker, user_id, added_at; unique ticker per profile.
positions	Ticker, quantity, average cost and updated timestamp.
trades	Append-only history: ticker, side, quantity, execution price, timestamp.
portfolio_snapshots	Total portfolio value and timestamp for the historical P&L; chart.
chat_messages	Role, content, serialized action result and timestamp.

API contract to freeze before UI work

Method + path	Responsibility
GET /api/health	Return health status; useful for container verification.
GET /api/watchlist	Return watchlist and current simulated prices.
POST /api/watchlist	Add a ticker after validation.
DELETE /api/watchlist/{ticker}	Remove a ticker from the watchlist.
GET /api/portfolio	Return cash, positions, current value and unrealized P&L.;
POST /api/portfolio/trade	Validate and perform a virtual market-style buy/sell.
GET /api/portfolio/history	Return time-stamped portfolio value snapshots.
GET /api/stream/prices	Stream price events through SSE.
POST /api/chat	Return assistant message plus validated action results.

Trade service validation

- Normalize and validate the ticker; reject malformed or unknown symbols according to the project policy.
- Require a positive quantity and a supported side (buy/sell).
- For a buy, compute cost from the current simulator price and reject if virtual cash is insufficient.
- For a sell, reject if the virtual portfolio does not own enough shares.
- Update cash, positions and trade history atomically so partial updates cannot leave a corrupt ledger.
- Return a typed result containing the fill price, updated portfolio summary and any validation error.

09 Price simulation and SSE streaming

The default market data source should be a local simulator. It is deterministic in tests and does not require a third-party data subscription. The project plan describes a GBM-style model with ticker-specific volatility and correlated movement; implement the simplest version that meets the visual and testing requirements before adding sophistication.

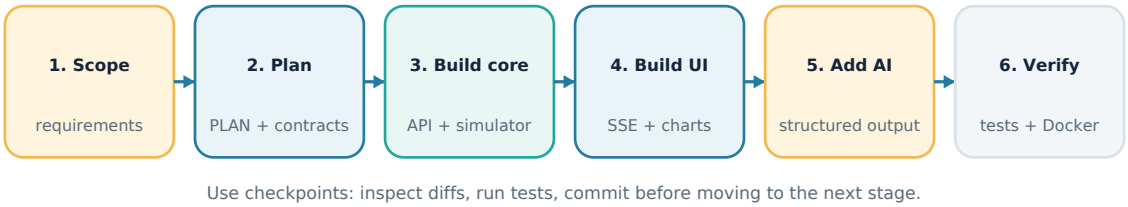


Figure 3. Recommended build sequence: plan, core API, interface, AI, verification.

How the price stream works

1. At application startup, initialize the seed watchlist and virtual starting cash if the database is empty.
2. A background coroutine updates each tracked ticker at a controlled interval and records the previous/current price and timestamp in an in-memory cache.
3. The SSE route yields a compact event for each update. A typical event contains ticker, current price, previous price, timestamp and direction.
4. The frontend opens EventSource to `/api/stream/prices`, updates the watchlist and chart state, and shows a connection status.
5. On disconnect, let EventSource retry and expose the reconnecting/disconnected state instead of silently freezing prices.

SSE payload example

```
event: price
data: {"ticker":"AAPL","price":190.42,"previous_price":190.11,
       "timestamp":"2026-10-19T12:00:00Z","direction":"up"}
```

Practical implementation notes

- Do not sleep-loop separately inside every client connection if one shared background price cache is sufficient.
- Keep one schema for simulated and optional external prices so the frontend cannot tell which adapter provided them.
- The default path should work without any market-data key. Keep external data optional and out of the first milestone.
- The price source should be clearly labeled as simulated in the UI; do not present it as an actual live quote.

10 Add the LLM assistant safely

The AI chat feature should be added after the manual virtual-trading path works. The backend loads the current simulated portfolio and recent chat history, asks the model for a structured response, validates its structure and every proposed action, calls the same portfolio/watchlist services used by the UI, then saves the response and action results.

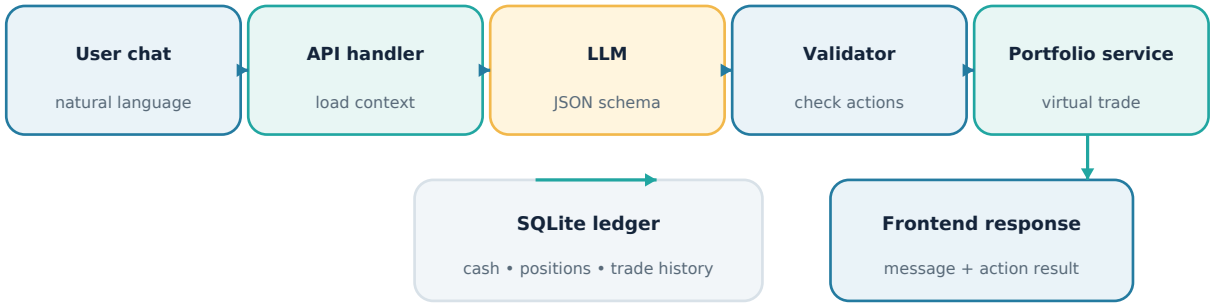


Figure 4. Chat-to-action pipeline. Every action passes through the backend validator.

Response contract

```
{
  "message": "Your portfolio is concentrated in a few positions.",
  "trades": [
    {"ticker": "AAPL", "side": "buy", "quantity": 2}
  ],
  "watchlist_changes": [
    {"ticker": "PYPL", "action": "add"}
  ]
}
```

Contract semantics

- `message` is required and shown in the chat panel.
- `trades` is optional. In this educational app, entries refer only to simulated portfolio transactions.
- `watchlist_changes` is optional and contains a ticker plus an add/remove action.
- Reject extra fields or malformed values where practical; return a safe error message instead of guessing what the model intended.
- Validate quantity, ticker, supported side, available virtual cash/shares, and the currently supported tool actions on the server.
- Record both the assistant response and any accepted/rejected actions so the user can understand what happened.

Recommended rollout

1. Create an LLM-independent Pydantic response model and tests first.
2. Implement a deterministic mock adapter and set `LLM_MOCK=true` for repeatable development.
3. Connect the chosen LLM provider through a small adapter, with the API key read from environment variables.
4. Prompt the model with portfolio context and a clear instruction to return only the agreed response schema.
5. Parse and validate output, then route approved simulated actions through the same domain services as manual trades.
6. Show the user the natural-language response, action success/failure, and resulting virtual portfolio state.

11 Build the frontend terminal

Build the dashboard around clear components and stable API types. Use mock JSON while designing the components, then wire them to the backend. The UI should make its data source and virtual-trading boundary obvious.

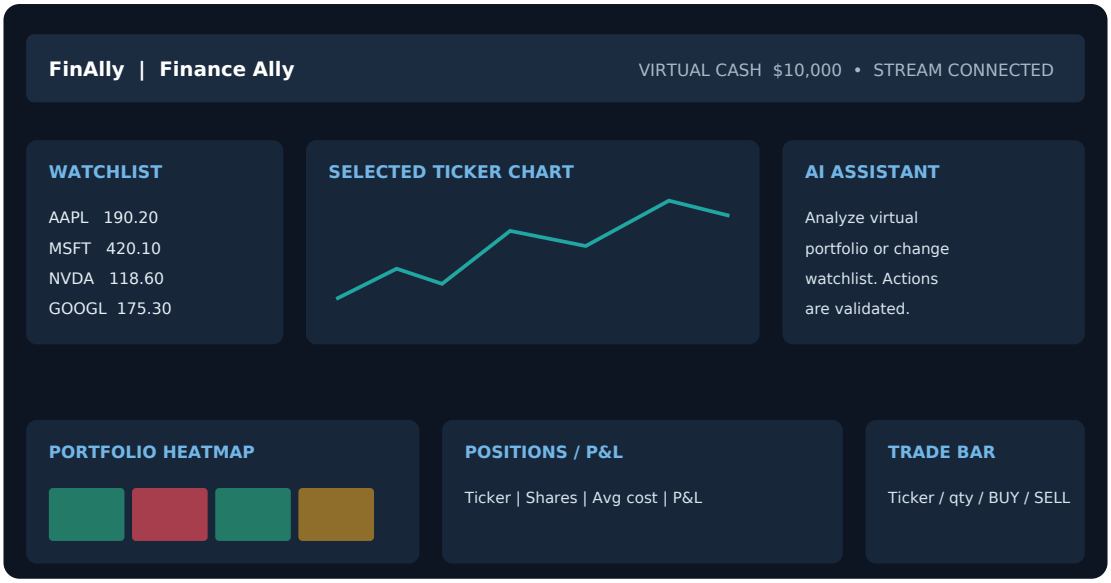


Figure 5. Conceptual dashboard wireframe (not a screenshot from the videos).

Required regions

Region	Behavior
Header	Total virtual portfolio value, virtual cash balance and stream connection status.
Watchlist	Ticker, simulated price, percentage change and compact sparkline; brief green/red price-change flash.
Selected-ticker chart	Clicking a watchlist row selects a larger price chart.
Portfolio heatmap	Each position gets a tile sized by portfolio weight and colored by simulated P&L.;
Portfolio history chart	Plot portfolio value from stored snapshots.
Positions table	Ticker, quantity, average cost, current price, unrealized P&L; and percentage change.
Trade bar	Ticker, quantity, simulated BUY and SELL buttons with clear validation feedback.
AI chat panel	Conversation history, input, loading state, and inline action receipts/errors.

Frontend implementation order

1. Create the page layout and empty components with typed props.
2. Add the API client module for REST paths from API_CONTRACT.md.
3. Add the EventSource hook; make connection state visible and clean up the connection on unmount.
4. Use a single source of truth for portfolio/watchlist state and refresh or reconcile it after actions.
5. Add charts and animations only after the data flow is stable.
6. Use empty, loading, error and disconnected states; avoid displaying stale prices as if they were current.

12 Integrate and run the application

During development, run each part on its own first; combine them after the API is stable.

Backend setup (from the backend folder)

```
cd backend
uv sync
uv run uvicorn app.main:app --reload --port 8000
```

The module path above is illustrative: confirm the actual entry-point module created by your agent and update the command to match it. Test `GET /api/health` before building the UI.

Frontend setup (from the frontend folder)

```
cd frontend
npm install
npm run dev
```

In development, Next.js may use its own port while FastAPI uses 8000. Configure the development proxy/base URL locally if needed. In production, static export is served by FastAPI on the same origin, so UI API calls use relative `/api/...` paths.

Environment configuration

```
# .env.example - safe placeholders only
OPENROUTER_API_KEY=replace_me_locally
MASSIVE_API_KEY=
LLM_MOCK=true
```

Copy the example to `.env` locally. Do not paste keys into prompts, commit them, or embed them in frontend code. Start with mock mode; only add a provider key when the backend schema and validation tests pass.

Docker verification path

```
# Build one image from the project root
docker build -t finally .

# Run it with a persistent SQLite volume
docker run --name finally-app -p 8000:8000 \
  --env-file .env -v finally-data:/app/db finally

# In a second terminal
curl http://localhost:8000/api/health

# Stop and remove the container (the named volume remains)
docker stop finally-app
docker rm finally-app
```

> For Windows PowerShell, the shell line-continuation character differs. Prefer the provided start script if the starter repository includes one; otherwise enter the Docker command on a single line or adapt the continuation syntax.

What the Dockerfile must do

- Stage 1: use a Node image, install frontend dependencies, and build/export the static frontend.
- Stage 2: use a Python image, install uv, sync locked backend dependencies, copy the backend plus static frontend output, and expose port 8000.
- Start Uvicorn so FastAPI serves API routes and static files in the same process/container.
- Mount a named volume to the runtime database directory so data persists across container replacement.
- Verify port 8000, environment variable loading, and volume path before claiming deployment is complete.

13 Testing, debugging and definition of done

The most reliable approach is layered: validate the deterministic business rules first, then the API, then the UI, then the packaged application. Avoid using real LLM calls as the only way to test application logic.

Test level	Must cover	Expected result
Unit - market	Simulator returns finite positive prices; price cache tracks previous/current values.	No invalid values; event direction matches price movement.
Unit - portfolio	Buy, sell, insufficient virtual cash, oversell, invalid quantity, unknown symbol.	Valid operation updates ledger once; invalid operation leaves state unchanged.
Unit - LLM parsing	Valid JSON, missing required message, wrong side, negative quantity, unexpected actions.	Valid responses parse; invalid actions are rejected safely.
API integration	Health, portfolio, watchlist, trade, history and chat endpoints.	Correct status codes and typed error responses.
Frontend	Loading, empty/error states, SSE connect/reconnect, price flash, watchlist edits, chat action receipt.	UI reflects backend truth and never silently hides failures.
End-to-end	Fresh start, seeded data, add/remove ticker, buy/sell, chat, restart persistence.	Full user journey works in mock mode.
Container	Build, start, health endpoint, refresh, stop/start, volume persistence.	App serves on localhost:8000 and state persists.

Recommended test commands (adjust to the generated repo)

```
# Backend tests (from backend/)
uv run pytest

# Frontend lint/build (from frontend/)
npm run lint
npm run build

# End-to-end tests (from test/ if Playwright is configured)
npm install
npx playwright test
```

Run `npm run` inside the frontend folder first to inspect the scripts that actually exist. Do not invent commands: if a script is absent, ask the coding agent to add it or document the correct alternative.

Independent reviewer prompt

```
Act as a read-only senior reviewer. Read planning/PLAN.md and
planning/API_CONTRACT.md first. Review the current diff for functional bugs,
security problems, invalid state transitions, inconsistent API types, missing tests,
and accidental scope expansion. Return findings ranked Critical/High/Medium/Low,
with file paths, line references, reproduction steps, and a suggested minimal fix.
Do not edit files or perform broad refactors.
```

> A passing build is not the same as a correct app. Confirm that invalid AI output and invalid manual trade requests cannot change the SQLite ledger. Confirm the UI labels the portfolio as simulated.

14 Course techniques: commands, sub-agents, hooks and plugins

The first lecture sequence around FiNALLY also demonstrates reusable coding-agent workflows. These are helpful but should not delay the core application. Add them after the project works end-to-end.

A. Custom slash command (saved prompt)

A project-level command can make a repeatable plan review available from the coding-agent UI. In the course, a Markdown file under `.claude/commands/` becomes a slash command; `$ARGUMENTS` inserts the argument typed after it.

```
# .claude/commands/doc-review.md
Review planning document named $ARGUMENTS. Append a section with
clarifying questions, contradictions, untested assumptions, and opportunities
to simplify. Do not delete requirements or change scope without asking.
Finish with a short prioritized list of decisions needed from the project owner.
```

Usage inside the coding agent

```
/doc-review PLAN.md
```

B. Reviewer sub-agent

A sub-agent runs in a separate context and reports a focused result, which keeps the main implementation conversation cleaner. Good independent jobs include reviewing a diff, checking the API contract, analyzing test failures, or scanning for leaked secrets. Avoid assigning two agents to edit the same file at the same time.

C. Hooks and events

A hook connects a tool lifecycle event to an action that should run automatically. Useful examples include running a format check after edits or reminding the agent to review before it stops. Hooks are an automation mechanism, not a substitute for tests or code review.

Concept	Best use in this project
PreToolUse / PostToolUse	Guard or log specific tool actions; keep restrictions narrow and inspectable.
Stop	Run or remind about a final verification checklist before the agent declares completion.
SessionStart / SessionEnd	Load or record task context; avoid storing secrets in logs.
SubagentStop	Collect review results or summarize a delegated task.

D. Skills and plugins

A skill packages reusable instructions or workflows. A plugin can bundle commands, skills, sub-agent definitions and hook configuration into one versioned unit for reuse across projects. The lecture demonstrates an independent reviewer plugin; for this capstone, keep any reviewer plugin optional until the core test suite works.

> Keep automation reviewable. Hooks must not silently run destructive commands, expose environment variables, approve all permissions, or push unreviewed changes. Use Git checkpoints and explicit approval for consequential actions.

15 Sandboxing, remote work and advanced agent workflows

Lectures 2 and 3 are supporting workflow lessons rather than new mandatory product requirements. They explain how to give coding agents a safer workspace, use remote execution, and manage larger tasks.

Sandboxing: use it to reduce the blast radius

A sandbox restricts what the agent can read or change and which commands it can run. The course describes local/native sandboxing and managed remote sandboxes. The exact UI and commands are tool-version-specific; consult current tool documentation instead of copying old commands blindly.

- Start with the project directory as the writable boundary.
- Grant network access only when the current task needs it, such as downloading dependencies.
- Do not put API keys in the repository or ask an agent to print them.
- Review the proposed permission set before using unattended or YOLO-style modes.
- Treat remote agent results like a pull request: inspect the diff, run tests, and merge only when ready.

Large-repository practices

- Maintain short, accurate root instructions and add local instruction files only where they help.
- Link to detailed documentation instead of pasting the entire architecture into every prompt.
- Use the same API contracts, formatting rules and test commands for all contributors.
- Prefer robust tests around behavior and public interfaces over brittle exact-text snapshots.
- Break work into small, reviewable patches with a clear acceptance test for each.

Agent SDK and desktop/workspace agents

The Agent SDK lesson distinguishes driving a coding agent programmatically from using a general-purpose agent framework. It is an advanced optional extension: only add it if you want to automate project tasks from Python. The course also surveys broader desktop/sidekick agents; they are not required to build FinAlly.

Which features are needed now?

Feature	Needed for v1?	Decision
Planning file + API contract	Yes	Create before parallel code.
One coding agent	Yes	Enough for the first backend/UI slice.
Parallel sub-agents	Optional	Add after file boundaries and contract are stable.
Custom reviewer command	Useful	Add after repeated reviews become tedious.
Hooks/plugins/marketplace	Optional	Package workflow once the core implementation is stable.
Sandbox / remote execution	Strongly recommended when available	Use least privilege and review diffs.
Agent SDK / extra sidekick tools	No	Exploratory; avoid adding complexity to the capstone.

16 Troubleshooting reference

Symptom	Likely cause	Action
Port 8000 is busy	Another local service/container is using it.	Stop the old container or map a different host port, e.g. 8001:8000.
UI loads but API calls fail	Wrong base URL, missing proxy, or API contract mismatch.	Inspect browser network console; use relative /api/* paths in production.
Prices freeze	SSE route not emitting, frontend listener disconnected, or background task stopped.	Check /api/stream/prices, browser EventSource errors and backend logs.
AI chat errors	Missing API key, provider routing changed, timeout, or schema mismatch.	Enable LLM_MOCK, test validation, then check current provider config and logs.
AI proposes invalid trades	Prompt alone is not a validator.	Reject invalid schema/domain actions; call the same portfolio service used by manual trades.
Cash/positions disagree	Updates were not transactional or duplicate action applied.	Use one service/transaction, add regression tests, and inspect trades ledger.
Database resets on restart	DB path is inside ephemeral container storage.	Mount named volume to the backend database path and verify path inside container.
Frontend build breaks in container	Static export configuration or copy path is wrong.	Check npm build output directory and Dockerfile copy path.
Agents overwrite files	Boundaries were not explicit or agents were launched too early.	Stop parallel edits, restore checkpoint, fix contract and ownership, rerun smaller tasks.
Agent finishes without test evidence	Prompt asked for implementation only.	Require test commands, exit status, changed-file summary and known limitations.

Useful debugging order

1. Read the terminal error and identify the failing layer.
2. Reproduce with a single deterministic input or mock response.
3. Inspect the relevant request/response schema and logs.
4. Fix the smallest owning component; avoid changes across unrelated folders.
5. Add a test for the failure, rerun the focused test, then rerun the full check set.

17 Build checklist and milestones

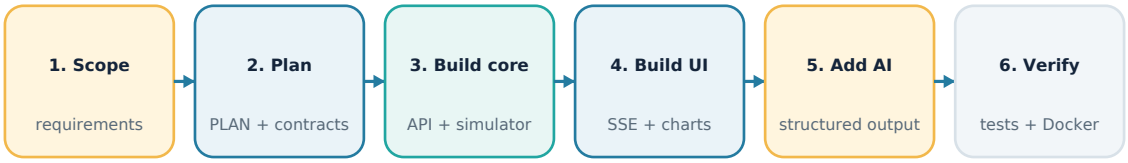


Figure 6. Keep milestones small and verifiable.

Milestone	Deliverable	Do not move on until...
M0: project contract	PLAN.md, CLAUDE.md/AGENTS.md, API_CONTRACT.md, Git baseline.	Scope, file ownership and data types are agreed.
M1: backend skeleton	Health endpoint, DB init/seed, portfolio GET.	API starts and tests pass.
M2: virtual trading	Buy/sell service, balances, positions and trades ledger.	Valid and invalid transaction tests pass.
M3: simulated prices	Price cache, simulator and SSE endpoint.	Stream updates and reconnect behavior are verified.
M4: frontend terminal	Watchlist, selected chart, positions, heatmap, trade bar.	UI matches API contract and shows loading/errors.
M5: LLM assistant	Mock mode, structured response model, validation and chat UI.	Invalid model output cannot mutate state.
M6: Docker + E2E	Multi-stage Docker image, persistent volume, tests.	Clean build runs at localhost:8000; restart preserves data.
M7: final review	Read-only reviewer, README, screenshots, known limitations.	Critical/high findings resolved; definition of done checked.

Final acceptance checklist

- [ ] The app explicitly labels prices and holdings as simulated.
- [ ] There are no real-money or brokerage endpoints.
- [ ] The plan and API contract remain aligned with the code.
- [ ] Manual virtual trades update cash, positions and history atomically.
- [ ] LLM responses are parsed as structured output and validated server-side.
- [ ] SSE recovers from disconnects and the UI surfaces connection state.
- [ ] SQLite data persists across container restarts using a named volume.
- [ ] Backend tests, frontend build/lint, and end-to-end scenarios pass.
- [ ] The Git diff has been reviewed; secrets and generated database files are ignored.
- [ ] README documents setup, environment variables, limitations and stop/start commands.

18 Glossary and source material

Quick glossary

Term	Meaning
Coding agent	A model-driven tool that can inspect a repository and perform software-development tasks with tools.
Sub-agent	A delegated worker using a separate context and returning a focused result.
Shared plan	A version-controlled file containing scope, architecture, boundaries and acceptance checks for all agents.
API contract	Stable request/response shapes that let frontend and backend work separately.
SSE	Server-Sent Events: a persistent one-way HTTP stream from server to browser.
Structured output	A model response constrained to a machine-readable schema such as JSON.
Mock mode	A deterministic substitute for external LLM calls, used for development and tests.
Sandbox	An execution boundary limiting what a coding agent can access or change.
Hook	An action triggered by an agent lifecycle event.
Plugin / marketplace	A packaged set of reusable commands, skills, agents and hooks, optionally distributed to other projects.
Virtual portfolio	A locally simulated ledger of cash, positions and trades; not connected to a broker.

Sources used to cross-check the visual lecture material

[1] Vibe Engineering — The FiNALLY project: your multi-agent capstone.
https://vibe-engineering-smit-web.vercel.app/class12-finally-project.html

[2] Vibe Engineering — Custom slash commands.
https://vibe-engineering-smit-web.vercel.app/class12-slash-commands.html

[3] Vibe Engineering — Agents and sub-agents.
https://vibe-engineering-smit-web.vercel.app/class12-subagents.html

[4] Vibe Engineering — Hooks.
https://vibe-engineering-smit-web.vercel.app/class12-hooks.html

[5] Vibe Engineering — Plugins and marketplaces.
https://vibe-engineering-smit-web.vercel.app/class12-plugins-marketplaces.html

[6] Vibe Engineering — Native sandbox.
https://vibe-engineering-smit-web.vercel.app/class13-native-sandbox.html

[7] Vibe Engineering — Claude Code on the Web.
https://vibe-engineering-smit-web.vercel.app/class13-claude-on-web.html

[8] Vibe Engineering — Working on large team codebases.
https://vibe-engineering-smit-web.vercel.app/class14-large-codebases.html

[9] Vibe Engineering — Driving Claude Code with the Agent SDK.
https://vibe-engineering-smit-web.vercel.app/class14-agent-sdk.html

[10] Public FinAlly project specification.
https://github.com/ed-donner/finally/blob/main/planning/PLAN.md

[11] Public FinAlly repository and quick-start reference.
https://github.com/ed-donner/finally